

PHPLogger

Logging strutturato per PHP 7.4+ - valutazione di adozione

Preparato da Michael Lavigna · 18 settembre 2026 · Documento interno di valutazione

In sintesi

PHPLogger produce log applicativi strutturati: una riga JSON per evento, su uno schema fisso e versionato, pensata per essere letta da una macchina prima che da una persona. La sua caratteristica distintiva non è il formato, che è ormai comune, ma il modo in cui tratta i dati sensibili: la protezione principale è strutturale e vale per costruzione, non affidata alla disciplina di chi scrive la chiamata.

4.200

righe di codice
di produzione

600

test automatici
94% di copertura

0

dipendenze
a runtime

~20 μ s

per record
~50.000 rec/s

~630 B

dimensione tipica
di un record

Valutazione in breve

Il nucleo è solido e insolitamente ben testato per un progetto di queste dimensioni: il codice di test supera quello di produzione, e la suite ha già ripagato il suo costo facendo emergere cinque difetti reali, tutti corretti.

Gli adattatori verso l'applicazione - PDO, cURL e i gestori di errore di PHP - **ora ci sono**, con i propri test, e si comportano in modo identico su PHP 7.4.3, 8.1 e 8.3. Era la voce di costo principale di un'eventuale adozione ed è stata chiusa.

Raccomandazione: adottabile per un servizio in cui il contenuto dei log è un tema di conformità o di sicurezza. Resta da decidere come distribuire e versionare la libreria, che è oggi il rischio operativo maggiore, e chi la mantiene.

1. Il problema che risolve

Il log applicativo PHP tipico è una stringa libera. Va bene per essere letto da una persona durante un incidente, molto meno per essere interrogato, aggregato o messo sotto allarme. Peggio: siccome il contenuto dipende da chi ha scritto la singola chiamata, è il posto in cui i dati personali finiscono per sbaglio, e da cui non escono più.

La libreria affronta tre problemi insieme:

- **Interrogabilità.** Un evento, una riga JSON, campi sempre negli stessi posti. Un collettore può indicizzarli senza espressioni regolari.
- **Stabilità nel tempo.** Lo schema è versionato in ogni riga e bloccato dai test: cambiare il nome, il tipo o l'ordine di un campo fa fallire la suite di proposito, perché è un cambiamento che rompe chi sta già leggendo quelle righe.
- **Contenimento dei dati sensibili.** Le strutture libere vengono scartate se non sono state esplicitamente permesse in configurazione. È il punto che distingue questa libreria dalle alternative, ed è trattato per esteso nella sezione 3.

2. Come funziona

Chi sviluppa sceglie un tipo di log e passa dati tipizzati. Non compone stringhe e non decide il formato di uscita. L'evento attraversa quattro stadi:

Stadio	Responsabilità
RecordFactory	Costruisce la busta e valida tutto ciò che arriva dal chiamante: nomi, livelli, esiti, durate. Da qui in poi il resto della catena può assumere un record ben formato.
SanitizingGate	Sostituisce le query SQL con la loro impronta, applica la lista di campi permessi, maschera i segreti riconoscibili nel testo libero, tiene il record sotto la dimensione massima.
Channel	Applica la soglia di livello e l'eventuale campionamento. Il routing è per tipo di log, con possibilità di forzare un canale.
Handler	Scrive su stream (stdout o file), su error_log, o scarta. Ogni destinazione reale sta dietro un circuit breaker.

Come si presenta un evento in uscita

```
{
  "schema_version": 1,
  "timestamp": "2026-09-18T09:14:22.481203+00:00",
  "level": "info",
  "service": "billing-api",
  "env": "production",
  "host": "web-01",
  "log_type": "db_query",
  "event": "db_query",
  "outcome": "success",
  "success": true,
  "duration": 0.0123,
  "error": null,
  "request_id": "req_51a81fd7d94a8561",
  "session_ref": null,
  "user_ref": "a3f19c22b7e10d44",
  "user_agent": null,
  "data": {
    "database": "production_db",
    "operation": "SELECT",
    "statement": "SELECT id, email FROM users WHERE tenant_id = ?",
    "statement_hash": "3e4e5c4be44ef130",
    "params": {
      "tenant_id": 4711,
      "_omitted": 2
    },
    "row_count": 42
  }
}
```

Due dettagli da notare in questa riga. **statement** non contiene i valori della query, solo la sua forma, e **statement_hash** permette di aggregare per forma invece che per stringa. **_omitted: 2** dice che due parametri sono stati scartati perché non permessi: l'omissione è visibile, non silenziosa.

3. Trattamento dei dati sensibili

Questo è il motivo principale per cui la libreria può interessarci, ed è anche la parte su cui vale la pena essere precisi, perché offre due livelli di protezione molto diversi fra loro. Confonderli porterebbe a fidarsi troppo.

3.1 Garanzia strutturale

Vale per costruzione: non ispeziona il contenuto, quindi non può sbagliarsi sul contenuto.

Meccanismo	Effetto
Lista di campi permessi	Ogni campo di tipo array sotto data (parametri, payload, contesto) viene scartato se le sue chiavi non sono elencate in configurazione. Nega per impostazione predefinita. Il numero di elementi scartati esce come _omitted .
Impronta delle query	Le istruzioni SQL escono normalizzate, con i valori sostituiti. Una query costruita per concatenazione non può perdere i suoi valori, anche se non avrebbe dovuto essere costruita così.
Stack trace senza argomenti	I frame sono ricostruiti da file, riga e funzione. Il metodo standard di PHP includerebbe gli argomenti della chiamata, che è uno dei modi più affidabili per far finire una password in un log.
Pseudonimi HMAC	Identificativi di utente e sessione escono come pseudonimi stabili e non reversibili. Senza chiave configurata sono semplicemente assenti: il valore in chiaro non viene mai emesso.

Meccanismo	Effetto
Cookie	Non vengono emessi in nessuna configurazione.

3.2 Mitigazione euristica

Sul testo libero (messaggi di eccezione, URL, user agent) agisce uno scrubber che maschera token Bearer, JWT, segreti nelle query string, indirizzi email, IBAN, codici fiscali e numeri di carta che passano il controllo di Luhn. Un segreto in una forma inattesa passa.

Cosa è emerso scrivendo i test, e perché conta

La suite ha fatto emergere due difetti reali proprio in questo secondo livello. Il primo: un segreto passato come primo parametro di una query string usciva **in chiaro**, mentre lo stesso segreto veniva correttamente mascherato altrove nella stessa riga. Il secondo: la regola contro le stringhe opache mascherava qualunque percorso di file, riducendo `/var/www/html/app/Http/Controllers/CheckoutController.php` a `/***.php` e rendendo illeggibile ogni stack trace in produzione.

Entrambi sono stati corretti e sono ora coperti da test. Il punto da portare a casa non è che la libreria fosse difettosa, ma **quale** dei due livelli ha ceduto: la garanzia strutturale ha retto, l'euristica no. È esattamente il comportamento che ci si deve aspettare, ed è il motivo per cui la seconda non va mai usata come unica linea di difesa.

4. Comportamento quando qualcosa si rompe

Il principio dichiarato, e verificato dai test, è che il logging non deve poter rompere il codice che osserva. Un'eccezione persa perché il logger ha fallito mentre la registrava sarebbe il caso peggiore.

Situazione	Comportamento
Configurazione non valida	Eccezione all'avvio. L'applicazione non parte: un logger mal configurato viene scoperto al deploy, non in produzione.
Evento costruito con dati non validi	Propagato in sviluppo e in CI, degradato in un record di servizio in produzione.
Sanificazione fallita	Record scartato ovunque. Un record non sanificato non viene mai scritto, nemmeno in forma ridotta.
Destinazione non disponibile	Errore catturato, il circuit breaker si apre e smette di insistere. L'applicazione non si ferma.
Errore mentre si registra un'eccezione	L'eccezione originale sopravvive sempre.
Errore sollevato dentro al logger	Un contatore di rientro impedisce che rientri nel logger attraverso il gestore di errori dell'applicazione.
Fallimento dentro un adattatore	Una query o una chiamata HTTP che è già avvenuta non diventa un errore applicativo solo perché il log non è stato scritto.

Le scritture parziali sono gestite esplicitamente: **fwrite** può scrivere solo parte del buffer, e una riga scritta a metà è un record corrotto per il collettore. La suite copre questo caso con una destinazione simulata che accetta un byte alla volta.

5. Prestazioni misurate

I numeri seguenti sono misurati, non stimati: PHP 7.4.3 in container, 20.000 iterazioni per riga, con scrittura reale della riga a destinazione. Coprono l'intero percorso: costruzione della busta, sanificazione, troncamento, codifica JSON e scrittura.

Operazione	Microsecondi (µs) per record	Record al secondo
Query di database	19,94	50.156
Chiamata HTTP in uscita	15,23	65.657
Eccezione con stack trace	15,37	65.042
Messaggio applicativo semplice	11,91	83.957
Query di database, senza scrittura	17,24	57.998
Riferimento: solo json_encode	0,52	1.936.655

Come leggere questi numeri

- **Il costo è quasi tutto CPU, non I/O.** La stessa operazione senza scrittura costa 17,24 µs contro 19,94 µs con scrittura: la sanificazione pesa circa sette volte la scrittura.
- **Le garanzie hanno un prezzo esplicito.** Un `json_encode` nudo della stessa struttura costa 0,52 µs. L'intera catena costa circa 38 volte tanto. Quel fattore compra validazione, impronta SQL, lista di campi permessi, mascheratura e controllo di dimensione.
- **In termini concreti:** una richiesta che registra 100 query di database paga circa 2 millisecondi. Su una richiesta che ne dura 200 è rumore; su un endpoint molto caldo, no.
- **La valvola di sfogo esiste.** Il campionamento è configurabile per tipo di log e non scarta mai i fallimenti e i record di livello error o superiore: taglia il costo dei record noiosi, non nasconde quelli interessanti.

Il costo dell'adattatore di database

Misurato confrontando la stessa query eseguita con PDO nudo e attraverso l'adattatore:

Operazione	µs per query	Query al secondo
PDO nudo, nessun logging	0,57	1.746.280
LoggingPdo, record scritto	18,32	54.595
Sovraccarico per query	17,74	-

Come non farsi ingannare da questo confronto

In termini relativi il sovraccarico è del **+3.099%**, che sembra catastrofico. Non lo è, ed è importante capire perché: il riferimento è una query SQLite in memoria che costa 0,57 µs, cioè una velocità che nessun database reale raggiunge.

Il numero che conta è quello assoluto: 17,7 µs per query. Su una query MySQL in rete da 1 millisecondo il sovraccarico è dell'1,8%; su una da 200 µs è dell'8,9%. Va valutato contro la latenza reale delle nostre query, non contro un caso limite di laboratorio.

6. Punti di forza

- **Nessuna dipendenza a runtime.** Nessuna superficie di supply chain, nessun conflitto di versioni con il resto dell'applicazione, nessun aggiornamento subito da terzi. In un contesto in cui l'introduzione di pacchetti va giustificata, è un vantaggio concreto.
- **La protezione principale è architetturale.** Negare per impostazione predefinita e richiedere una lista esplicita è una scelta corretta: sposta il problema da 'ricordarsi di non loggare' a 'decidere una volta cosa si logga'.
- **Gli adattatori sono pronti e testati.** PDO, cURL e i gestori di errore di PHP, con comportamento verificato identico su 7.4.3, 8.1 e 8.3.
- **Il contratto di uscita è bloccato dai test.** Chi costruisce cruscotti sopra queste righe non se le vede cambiare sotto i piedi senza che qualcuno lo abbia deciso.
- **Il logging non può fermare l'applicazione.** Guardia sull'intera catena, percorso di emergenza a volume limitato, circuit breaker, protezione dal rientro.
- **Qualità verificabile.** 600 test, 94% di copertura delle righe, esecuzione in ordine casuale per impedire dipendenze fra test, suite che gira sul runtime minimo dichiarato.

7. Limiti e rischi

Elencati con la severità che avrebbero per noi in caso di adozione, non in astratto.

Severità	Limite	Implicazione
Alta	Distribuzione per copia	Nessuna pubblicazione su Packagist: la libreria si installa copiando la cartella. Gli aggiornamenti sono manuali e, se più servizi la usano, le copie divergono. È oggi il rischio operativo maggiore e serve una decisione esplicita su versioning e propagazione delle correzioni.
Media	Nessuna versione rilasciata	Non esistono ancora tag di versione, e una modifica recente ha cambiato i nomi delle interfacce. Chi la adottasse oggi non ha un riferimento stabile a cui ancorarsi.
Media	Il costo di configurazione è reale	La lista di campi permessi è la garanzia, ma finché non viene compilata parametri, contesto e payload escono vuoti. Il valore del log dipende da un lavoro di analisi iniziale che qualcuno deve fare.
Media	Scritture sincrone e bloccanti	La scrittura avviene sul percorso della richiesta. Non c'è accodamento né invio asincrono. Il circuit breaker protegge dal caso peggiore, ma il costo resta a carico della richiesta.
Media	Un solo autore	Progetto personale, nessuna comunità. In caso di adozione la manutenzione ricade su di noi: va individuato chi la prende in carico.
Bassa	Un solo nome evento per connessione	L'adattatore PDO registra tutte le query sotto lo stesso nome evento. Distinguere "user_lookup" da "invoice_fetch" richiede di costruire il payload a mano o una seconda istanza.
Bassa	PSR-3 non dichiarato formalmente	Le firme sono compatibili, ma l'interfaccia non è implementata perché non installabile senza Composer. Serve una sottoclasse di poche righe per le librerie di terze parti.
Bassa	Mascheratura euristica fallibile	Lo scrubber è una mitigazione, non una garanzia, e si è già dimostrato fallibile due volte. Va comunicato a chi lo usa, perché il rischio vero è che venga scambiato per una garanzia.
Bassa	Nessuna pipeline di CI	Esiste l'ambiente Docker che esegue la suite sul runtime minimo, ma nessuna automazione che lo faccia a ogni modifica. È mezza giornata di lavoro.

Una nota di contesto, esterna alla libreria

PHP 7.4 è fuori supporto dal novembre 2022. La libreria lo supporta deliberatamente, il che è un vantaggio se il nostro parco applicativo è ancora lì. Ma se lo è, il rischio rilevante per la sicurezza non è il logger: è l'interprete. Vale la pena separare le due discussioni e non lasciare che questa scelta tecnica faccia sembrare accettabile la prima.

8. Confronto con l'alternativa ovvia

La domanda naturale è perché non usare Monolog, che è lo standard di fatto.

Aspetto	Monolog	PHPLogger
Diffusione e ecosistema	Standard di fatto, decine di destinazioni pronte, PSR-3 nativo, mantenuto da molti.	Nessun ecosistema, un solo autore.
Dati sensibili	Nessuna protezione strutturale. I processor sono facoltativi e in genere euristici.	Negazione per impostazione predefinita sulle strutture libere; impronta SQL; pseudonimi.
Stabilità del formato	Dipende dal formatter scelto e dalla disciplina del team.	Schema versionato e bloccato dai test.
Dipendenze	Richiede Composer e trascina le proprie dipendenze.	Nessuna.
Tempo per il primo log utile	Ore.	Ore: gli adattatori ora sono inclusi.

La sintesi onesta: Monolog è la scelta giusta se il problema è *avere* dei log. PHPLogger diventa interessante se il problema è *cosa c'è dentro* ai log, e se quella domanda ha una rilevanza di conformità o di sicurezza. Vale anche la pena sapere che una parte del valore di PHPLogger si può ottenere aggiungendo processor a Monolog: si perde la garanzia strutturale, si guadagna tutto l'ecosistema.

9. Come si integra

Gli adattatori sono la parte che tocca l'applicazione, e sono l'unica che sa qualcosa di PDO, di cURL o dei meccanismi di errore di PHP. Il giudizio che non possono dare da soli - questa operazione è riuscita? - viene sempre iniettato.

Database

```
$pdo = new LoggingPdo($dsn, $user, $password,
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION], $logger, 'production_db');

$pdo->prepare('SELECT id FROM users WHERE tenant_id = ?')->execute([$tenantId]);
```

È una sottoclasse di PDO, non un decoratore: il codice esistente che dichiara **PDO** nelle firme continua a funzionare senza modifiche. Servono quattro punti di intercettazione, non uno: SQLite e PostgreSQL rifiutano una query malformata già dentro prepare(), quindi execute() non viene mai raggiunto. I fallimenti sono registrati **prima** che l'eccezione venga rilanciata, altrimenti con ERRMODE_EXCEPTION si perderebbero proprio i record che contano.

Chiamate HTTP in uscita

```
$recorder = new CurlRecorder($logger);           // oppure con una policy propria
$body = $recorder->execute($handle, 'GET');      // sostituisce curl_exec()
```

Tutte le metriche di temporizzazione arrivano da curl_getinfo(), quindi il chiamante non misura nulla. Se un 404 sia un fallimento lo decide una policy sostituibile.

Errori di PHP

```
(new ErrorHandlerBridge($logger))->register();
```

Tre agganci, perché PHP segnala i guasti in tre modi diversi. Non viene inghiottito niente: i gestori preesistenti vengono concatenati e un'eccezione non catturata viene rilanciata, così il processo muore come PHP prevede. Un buffer di memoria viene riservato all'avvio, perché un fatale da memoria esaurita non ne lascia per scrivere il record che lo descrive.

Decidere cosa è permesso uscire

```
'capture' => [
    'fields' => [
        'db_query'      => ['params' => ['tenant_id', 'status']],
        'service_call' => ['timing'  => ['dns', 'connect', 'ttfb', 'total']],
        'php_log'       => ['context' => ['route', 'job_id']],
    ],
],
```

Tutto ciò che non compare qui viene scartato e conteggiato in **_omitted**. È il file su cui si concentra la decisione di quali dati è accettabile registrare, ed è il punto in cui una revisione di conformità ha senso.

10. Stato e maturità

Voce	Stato
Codice di produzione	65 file, circa 4.200 righe non vuote.
Codice di test	56 file, circa 5.000 righe: più test che produzione.
Suite	600 test, 1.359 asserzioni, 94,0% di copertura delle righe.
Ordine di esecuzione	Casuale, per impedire che un test dipenda da un altro.
Runtime verificati	PHP 7.4.33 in locale e PHP 7.4.3 in container per la suite; gli adattatori anche su 8.1 e 8.3.
Ambiente riproducibile	docker compose up costruisce tutto ed esegue la suite senza nulla installato sulla macchina.
Contratto di uscita	Bloccato da test dedicati su nomi, tipi e ordine dei campi.
Percorsi fatali	Errore fatale, memoria esaurita ed eccezione non catturata sono testati in processi figli reali.
Analisi statica	Assente. PHPStan o Psalm sarebbero un'aggiunta naturale.
Integrazione continua	Assente. L'ambiente per eseguirla c'è già.

11. Cosa manca, in ordine di importanza

#	Intervento	Sforzo indicativo
1	Tag di versione e politica di aggiornamento delle copie	Meno di un giorno
2	Compilazione della lista dei campi permessi per i nostri casi d'uso	Dipende dal servizio
3	Pipeline di integrazione continua che esegua la suite a ogni modifica	Mezza giornata
4	Analisi statica in pipeline	Mezza giornata
5	Consegna asincrona, se i volumi lo richiederanno	Da valutare

12. Raccomandazione

Ha senso adottarlo se:

- Il contenuto dei log è per noi un tema di conformità o di sicurezza, e non solo di diagnostica.
- Possiamo indicare una persona responsabile della manutenzione.
- Accettiamo di gestire la distribuzione per copia con una disciplina esplicita.

Non ha senso adottarlo se:

- Il contenuto dei log non è un problema aperto: in quel caso Monolog costa meno e rende di più.
- I volumi sono tali da non poter assorbire il costo per record e non possiamo campionare.
- Nessuno può prenderlo in carico: senza un responsabile diventa codice orfano in un punto critico.

Proposta di prossimo passo

Una prova su **un solo servizio non critico**: collegare l'adattatore PDO, compilare la lista dei campi permessi per quel servizio e tenerlo in esercizio per qualche settimana.

Rispetto alla valutazione precedente il costo di questa prova è sceso in modo sostanziale, perché gli adattatori non vanno più scritti. Quello che resta da misurare è il lavoro di configurazione e, soprattutto, se i log che ne escono sono effettivamente più utili di quelli attuali. Il rischio è contenuto: per costruzione un guasto del logger non ferma l'applicazione.

I dati di prestazione in questo documento sono stati misurati il 18 settembre 2026 su PHP 7.4.3 in container, 20.000 iterazioni per misura. I dati su test e copertura provengono dall'esecuzione della suite sullo stesso runtime.